

Deep Learning for NLP

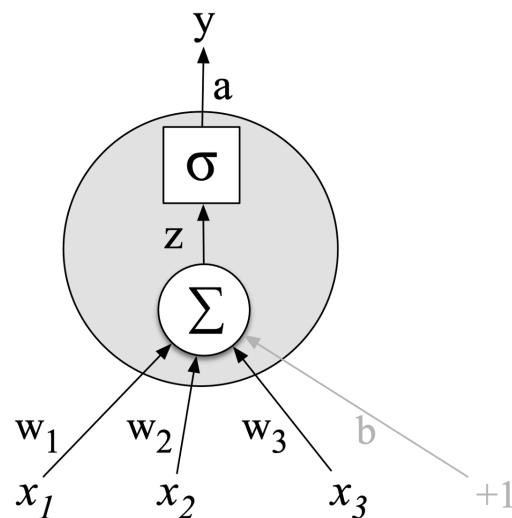
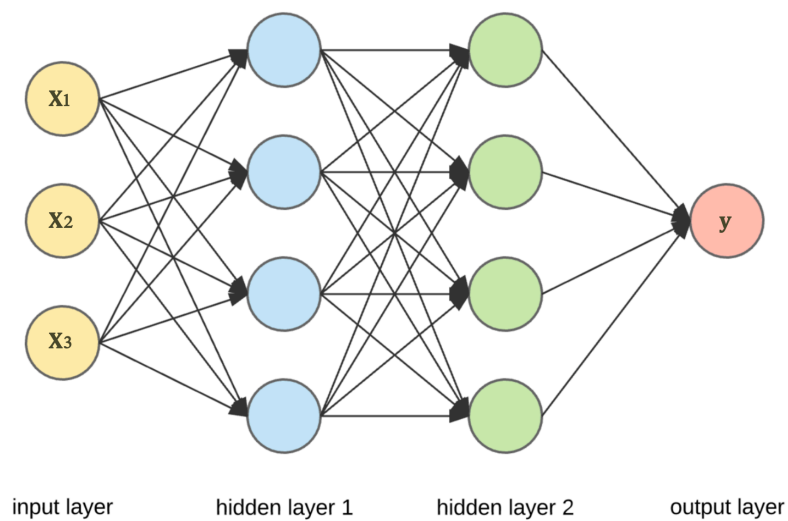
Deep Learning is a type of Machine Learning that uses **Neural Networks with multiple layers** to learn patterns from data.

Deep Learning can be used in NLP tasks such as:

- Text Classification
- Sentiment Analysis
- Question Answering

In this lab, we will build a **deep learning model** to classify Yelp restaurant reviews as:

- **0 = Negative**
- **1 = Positive**



1. Load the Yelp Dataset

Each row contains:

- a restaurant review
- its sentiment label

```
%pip install pandas scikit-learn

import pandas as pd

df = pd.read_csv("07-yelp-dataset.txt", sep="\t", names=["review", "label"])

print("Number of reviews:", len(df))
df.head()
```

Requirement already satisfied: pandas in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (3.0.0)
Requirement already satisfied: scikit-learn in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (1.8.0)
Requirement already satisfied: numpy>=1.26.0 in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from pandas) (2.4.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /Users/almadani/Library/Python/3.13/lib/python/site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: scipy>=1.10.0 in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from scikit-learn) (1.17.1)
Requirement already satisfied: joblib>=1.3.0 in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.2.0 in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: six>=1.5 in /Users/almadani/Library/Python/3.13/lib/python/site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

[notice] A new release of pip is available: 25.2 -> 26.2.1

[notice] To update, run: /Users/almadani/Desktop/study/nlp/nlp lab/.venv/bin/python -m pip install --upgrade pip

Note: you may need to restart the kernel to use updated packages.

Number of reviews: 1000

	review	label
0	Wow... Loved this place.	1
1	Crust is not good.	0
2	Not tasty and the texture was just nasty.	0
3	Stopped by during the late May bank holiday of...	1
4	The selection on the menu was great and so wer...	1

```
print(df["label"].value_counts())
```

```
label
1    500
0    500
Name: count, dtype: int64
```

2. Preprocessing the Dataset

```
import re

def preprocess_text(text):

    # 1. Convert to lowercase
    text = text.lower()

    # 2. Remove punctuation and special characters
    text = re.sub(r'[^a-zA-Z\s]', '', text)

    # 3. Remove extra spaces
    text = re.sub(r'\s+', ' ', text).strip()

    return text

df["review"] = df["review"].apply(preprocess_text)
df.head()
```

	review	label
0	wow loved this place	1
1	crust is not good	0
2	not tasty and the texture was just nasty	0
3	stopped by during the late may bank holiday of...	1
4	the selection on the menu was great and so wer...	1

3. Split the Data

We divide the dataset into:

- **80% training data** — used to train the model
- **20% testing data** — used to evaluate the model

`stratify=y` keeps the proportion of positive and negative reviews similar in both sets.

```
from sklearn.model_selection import train_test_split

X = df["review"]
y = df["label"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)

print("Training reviews:", len(X_train))
print("Testing reviews:", len(X_test))
```

```
Training reviews: 800
Testing reviews: 200
```

4. Convert Text to TF-IDF Vectors

A neural network cannot directly understand text.

TF-IDF converts each review into a vector of numbers.

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(max_features=1000, lowercase=True)

X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

print("Training TF-IDF shape:", X_train_tfidf.shape)
print("Testing TF-IDF shape:", X_test_tfidf.shape)
```

```
Training TF-IDF shape: (800, 1000)
Testing TF-IDF shape: (200, 1000)
```

5. Convert the Data to PyTorch Tensors

PyTorch neural networks work with **tensors**.

```
%pip install torch

import torch
import torch.nn as nn

torch.manual_seed(42)

# Convert TF-IDF data to PyTorch tensors
X_train_tensor = torch.FloatTensor(X_train_tfidf.toarray())
X_test_tensor = torch.FloatTensor(X_test_tfidf.toarray())

y_train_tensor = torch.FloatTensor(y_train.to_numpy(copy=True)).reshape(-1, 1)
```

Requirement already satisfied: torch in /Users/almadani/Desktop/study/nlp/nlp lab/.venv/lib/python3.13/site-packages (2.14.0)
Requirement already satisfied: filelock in /Users/almadani/Library/Python/3.13/lib/python/site-packages (from torch) (3.32.2)
Requirement already satisfied: typing-extensions>=4.10.0 in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools>=77.0.3 in /Users/almadani/Desktop/study/nlp/nlp lab/.venv/lib/python3.13/site-packages (from torch) (84.0.0)
Requirement already satisfied: sympy>=1.13.3 in /Users/almadani/Desktop/study/nlp/nlp lab/.venv/lib/python3.13/site-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from torch) (3.6.1)
Requirement already satisfied: Jinja2 in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /Users/almadani/Library/Python/3.13/lib/python/site-packages (from torch) (2026.7.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /Users/almadani/Desktop/study/nlp/nlp lab/.venv/lib/python3.13/site-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from Jinja2->torch) (3.0.3)

[notice] A new release of pip is available: 25.2 -> 26.2.1
[notice] To update, run: /Users/almadani/Desktop/study/nlp/nlp lab/.venv/bin/python -m pip install --upgrade pip

Note: you may need to restart the kernel to use updated packages.

6. Build a Simple Neural Network

Our network has:

1. **Input layer** — receives the TF-IDF features
2. **Hidden layer** — learns useful patterns
3. **ReLU activation** — adds non-linearity
4. **Output layer** — produces one value for binary classification

We do **not** put `Sigmoid` inside the model because `BCEWithLogitsLoss` applies it internally in a numerically stable way.

```
input_size = X_train_tensor.shape[1]
# torch.size([800, 1000])

model = nn.Sequential(
    nn.Linear(input_size, 64), # Input layer
    nn.ReLU(),

    nn.Linear(64, 32), # Hidden layer
    nn.ReLU(),

    nn.Linear(32, 1) # Output layer
)

print(model)
```

```
Sequential(
  (0): Linear(in_features=1000, out_features=64, bias=True)
  (1): ReLU()
  (2): Linear(in_features=64, out_features=32, bias=True)
  (3): ReLU()
  (4): Linear(in_features=32, out_features=1, bias=True)
)
```

7. Choose the Loss Function and Optimizer

- **BCEWithLogitsLoss** is used for binary classification. To measure the model's mistakes.
- **Adam** is responsible for changing the model's weights so that the loss becomes smaller.

```
loss_function = nn.BCEWithLogitsLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

8. Train the Neural Network

One **epoch** means the model has seen the full training dataset once.

For every epoch:

1. Make predictions
2. Calculate the loss
3. Calculate gradients with backpropagation
4. Update the weights

```
epochs = 10

for epoch in range(epochs):
    model.train()
    optimizer.zero_grad()

    # 1. Forward pass
    outputs = model(X_train_tensor)

    # 2. Calculate loss
    loss = loss_function(outputs, y_train_tensor)

    # 3. Backpropagation
    loss.backward()

    # 4. Update model weights
    optimizer.step()

    if (epoch + 1) % 5 == 0:
        print(f"Epoch {epoch + 1}/{epochs} - Loss: {loss.item():.4f}")
```

```
Epoch 5/10 - Loss: 0.6316
Epoch 10/10 - Loss: 0.3809
```

9. Evaluate the Model

The model outputs raw scores called **logits**.

We apply `sigmoid` to convert them to probabilities between 0 and 1.

- Probability ≥ 0.5 → **Positive (1)**
- Probability < 0.5 → **Negative (0)**

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

model.eval()

with torch.no_grad():
    test = model(X_test_tensor)
    test_probabilities = torch.sigmoid(test)
    predictions = (test_probabilities >= 0.5).int().numpy().flatten()

accuracy = accuracy_score(y_test, predictions)

print(f"Test Accuracy: {accuracy:.3f}")
```

```
Test Accuracy: 0.745
```

10. Classification Report and Confusion Matrix

The classification report shows:

- **Precision**
- **Recall**
- **F1-score**

The confusion matrix shows how many reviews were classified correctly and incorrectly.

```
print("Classification Report:")
print(classification_report(y_test, predictions, target_names=["Negative", "Positive"]))

print("Confusion Matrix:")
print(confusion_matrix(y_test, predictions))
```

```
Classification Report:
              precision    recall  f1-score   support

   Negative         0.74        0.73        0.73         96
   Positive         0.75        0.76        0.76        104

   accuracy                   0.74         200
  macro avg         0.74        0.74        0.74         200
 weighted avg         0.74        0.74        0.74         200
```

```
Confusion Matrix:
[[70 26]
 [25 79]]
```

Task 1 – Represent Reviews Using Word2Vec

In the previous part of the lab, TF-IDF was used to convert each review into numerical features. Now, use Word2Vec instead.

Requirements: Tokenize the training and testing reviews. Train a Word2Vec model using the training reviews. Use: vector_size = 200 window = 5 min_count = 1 sg = 1

Convert each review into a single vector. Convert the resulting vectors into PyTorch tensors.

```
%pip install gensim numpy

import numpy as np
from gensim.models import Word2Vec

train_tokens = [review.split() for review in X_train]
test_tokens = [review.split() for review in X_test]

word2vec = Word2Vec(
    sentences=train_tokens,
    vector_size=200,
    window=5,
    min_count=1,
    sg=1,
    seed=42,
    workers=1
)

def review_vector(tokens):
    vectors = [word2vec.wv[word] for word in tokens if word in word2vec.wv]
    if vectors:
        return np.mean(vectors, axis=0)
    return np.zeros(200, dtype=np.float32)

X_train_w2v = np.array([review_vector(tokens) for tokens in train_tokens])
X_test_w2v = np.array([review_vector(tokens) for tokens in test_tokens])

X_train_w2v_tensor = torch.tensor(X_train_w2v, dtype=torch.float32)
X_test_w2v_tensor = torch.tensor(X_test_w2v, dtype=torch.float32)
y_train_w2v_tensor = torch.tensor(y_train.values, dtype=torch.float32).reshape(-1, 1)
y_test_w2v_tensor = torch.tensor(y_test.values, dtype=torch.float32).reshape(-1, 1)

print("Training Word2Vec shape:", X_train_w2v_tensor.shape)
print("Testing Word2Vec shape:", X_test_w2v_tensor.shape)
```

Requirement already satisfied: gensim in /Users/almadani/Desktop/study/nlp/nlp lab/.venv/lib/python3.13/site-packages (4.4.0)
Requirement already satisfied: numpy in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (2.4.0)
Requirement already satisfied: scipy>=1.7.0 in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from gensim) (1.17.1)
Requirement already satisfied: smart_open>=1.8.1 in /Users/almadani/Desktop/study/nlp/nlp lab/.venv/lib/python3.13/site-packages (from gensim) (8.0.1)
Requirement already satisfied: wrapt in /Users/almadani/Desktop/study/nlp/nlp lab/.venv/lib/python3.13/site-packages (from smart_open>=1.8.1->gensim) (2.4.1)

[notice] A new release of pip is available: 25.2 -> 26.2.1

[notice] To update, run: /Users/almadani/Desktop/study/nlp/nlp lab/.venv/bin/python -m pip install --upgrade pip

Note: you may need to restart the kernel to use updated packages.

Training Word2Vec shape: torch.Size([800, 200])
Testing Word2Vec shape: torch.Size([200, 200])

Task 2 – Build a Different Neural Network

Use the Word2Vec vectors from Task 1 as the input to a new neural network. Your network must contain: An input layer Three hidden layers ReLU activation functions One output neuron Use the following architecture:

Input

↓

128 neurons

↓

64 neurons

↓

32 neurons

↓

1 output neuron

Train the model using: BCEWithLogitsLoss , Adam optimizer , Learning rate = 0.001 , 15 epochs. Then calculate the test accuracy.

```

torch.manual_seed(42)

w2v_model = nn.Sequential(
    nn.Linear(200, 128),
    nn.ReLU(),
    nn.Linear(128, 64),
    nn.ReLU(),
    nn.Linear(64, 32),
    nn.ReLU(),
    nn.Linear(32, 1)
)

w2v_loss_function = nn.BCEWithLogitsLoss()
w2v_optimizer = torch.optim.Adam(w2v_model.parameters(), lr=0.001)

for epoch in range(15):
    w2v_model.train()
    w2v_optimizer.zero_grad()
    outputs = w2v_model(X_train_w2v_tensor)
    loss = w2v_loss_function(outputs, y_train_w2v_tensor)
    loss.backward()
    w2v_optimizer.step()

    if (epoch + 1) % 5 == 0:
        print(f"Epoch {epoch + 1}/15 - Loss: {loss.item():.4f}")

w2v_model.eval()
with torch.no_grad():
    probabilities = torch.sigmoid(w2v_model(X_test_w2v_tensor))
    w2v_predictions = (probabilities >= 0.5).int().numpy().flatten()

print(f"Test Accuracy: {accuracy_score(y_test, w2v_predictions):.3f}")

```

```

Epoch 5/15 - Loss: 0.6947
Epoch 10/15 - Loss: 0.6935
Epoch 15/15 - Loss: 0.6933
Test Accuracy: 0.480

```